



INSTITUTE FOR ADVANCED COMPUTING AND SOFTWARE DEVELOPMENT, AKURDI, PUNE

"EventMitra"

An Event Ticketing and Management Platform

PGCP-AC — February 2026

Submitted By:

Group No.58

Roll. No.	Name Of Student
262113	Suryansh Pratap Singh
262169	Yash Chaudhari

Mrs.Manjiri Deshpande

Project Guide

Mr. Anil Sharma

Centre Coordinator

ABSTRACT

EventMitra is a web-based event ticket booking and management platform developed to simplify the complete lifecycle of event discovery, ticket selection, booking, online payment, receipt generation and refund processing. The application provides separate role-based capabilities for Administrators, Organizers and Attendees. Attendees can register, verify their mobile/email through OTP, browse events, select tickets, create bookings and complete payments. Organizers can create and manage events, manage ticket inventory and monitor event performance through a dashboard. Administrators manage users, payments and refund approvals

The backend is implemented using Java 21 and Spring Boot 3.3.5 with Spring Data JPA, Hibernate, Spring Security, JWT authentication, validation and MySQL. Razorpay is integrated for payment order creation, signature verification and refunds. A separate .NET 8 notification service provides OTP generation, email delivery and audit logging. The React 19 frontend, built with Vite, communicates with the backend through REST APIs. Event images are stored through a local file-storage component. The project also contains an AI assistant using Spring AI with an OpenAI-compatible Groq endpoint and Llama 3.3 70B model for EventMitra-related assistance.

The database is designed around eight core application tables—users, event, ticket, booking, payment, receipt, refund and otp_verification—with foreign-key relationships supporting the booking and payment workflow. The resulting system provides a modular, secure and practical platform for digital event ticket management.

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to the Institute for Advanced Computing and Software Development (IACSD), Akurdi, Pune, for providing the learning environment, infrastructure and guidance required to complete this project successfully.

I am thankful to my project guide and faculty members for their continuous guidance, technical suggestions and valuable feedback throughout the design and implementation of EventMitra. Their support helped me understand practical aspects of REST API development, database design, authentication, payment integration and frontend-backend communication.

I also acknowledge the support of my classmates, friends and family members for their encouragement during the development and testing phases of the project. Finally, I am grateful for the opportunity to work on a project that allowed me to integrate Java, Spring Boot, React, MySQL, payment services and a supporting notification service into one complete application.

Suryansh Pratap Singh(260241220190)

Yash Chaudhari(260241220217)

TABLE OF CONTENTS

Sr. No.	Description	Page No.
1	Introduction	4
1.1	Purpose, Scope and Objectives	5
1.2	System Roles and Functionalities	5
2	Software Requirement Specification	7
2.1	Functional Requirements	7
2.2	Non-Functional Requirements and Environment	7
3	System Architecture	9
4	Diagrams	10
4.1	ER Diagram	10
4.2	Use Case Diagram	11
4.3	Data Flow Diagram	12
4.4	Activity Diagram	13
4.5	Class Diagram	15
4.6	Sequence Diagram	17
5	Database Design	18
5.1	Schema Overview	18
5.2	Table Structures	19
5.3	Database Relationships	21
6	API Design and Security	23
7	Snapshots	24
8	Testing and Validation	35
9	Conclusion	37
10	References	37

1. INTRODUCTION

EventMitra is a web-based event registration and ticket management application that provides a single platform for attendees and event organizers. It centralizes event discovery, ticket selection, booking, payment, receipt generation and refund processing.

The system uses a React frontend and a Spring Boot REST backend connected to MySQL. The backend contains controller, service, repository, entity, DTO, security and exception-handling layers. Razorpay provides payment processing, while a separate .NET 8 notification service provides OTP, email and audit-log capabilities.

EventMitra is designed to solve common event-ticketing problems such as fragmented booking workflows, inaccurate ticket availability and manual payment/receipt handling. The application uses validation, role-based access control and relational data relationships to keep the workflow consistent and traceable.

1.1 PURPOSE, SCOPE AND OBJECTIVES

Purpose

The purpose of EventMitra is to provide a centralized web platform where users can discover events and purchase tickets, while organizers can publish events and manage ticket inventory. The platform also supports OTP verification, online payment, receipt generation and controlled refunds.

Scope

- Registration, OTP verification, login and role-based access.
- Event and ticket CRUD operations with event image upload.
- Event browsing and filtering by category/location.
- Booking with ticket availability validation and PENDING status.
- Razorpay order creation, signature verification and refunds.
- Automatic receipts, confirmation emails and organizer dashboard.
- Admin refund approval/rejection and AI-based EventMitra assistance.

Objectives

1. Simplify online event discovery and ticket booking.
2. Maintain accurate ticket inventory.
3. Secure user authentication and authorization.
4. Integrate online payments and refunds.
5. Automate receipts and notifications.
6. Provide a maintainable MySQL-backed full-stack application.

1.2 SYSTEM ROLES AND FUNCTIONALITIES

Role	Responsibilities	Examples
ADMIN	Platform and financial management.	Users, payments, refunds, events and tickets.
ORGANIZER	Event and ticket management.	Create/update/delete events, upload images, dashboard.
ATTENDEE	Event participation and booking.	Register, browse, book, pay, view bookings, request refund.

Major Functional Modules

Module	Functionality
Authentication	Registration, OTP verification, BCrypt password hashing and JWT login.
Event & Ticket	CRUD operations, filters, image upload and inventory management.
Booking & Payment	Availability validation, Razorpay order creation and signature verification.
Receipt & Refund	Receipt generation, refund request and admin approval/rejection.
Dashboard & AI	Organizer metrics and EventMitra conversational assistance.

Technology Stack

Layer	Technology
Frontend	React 19, Vite, React Router, Axios, HTML/CSS
Backend	Java 21, Spring Boot 3.3.5, Spring Web, Spring Data JPA, Hibernate
Security / DB	Spring Security, JWT, BCrypt, MySQL 8
Integration	Razorpay, .NET 8 Notification Service, Spring AI / Groq

2. SOFTWARE REQUIREMENT SPECIFICATION

2.1 FUNCTIONAL REQUIREMENTS

ID	Requirement	Description
FR-01	Registration & OTP	Register users, send a six-digit OTP and activate the account after successful verification.
FR-02	Authentication	Authenticate credentials and issue a JWT token with role information.
FR-03	Event Management	ADMIN/ORGANIZER users create, update and delete events; users can browse/filter events.
FR-04	Ticket Management	ADMIN/ORGANIZER users manage ticket price and quantity for events.
FR-05	Booking	ATTENDEE/ADMIN users create bookings only when requested quantity is available.
FR-06	Payment	Create Razorpay orders and verify signatures before confirming a booking.
FR-07	Receipt & Notification	Generate a receipt and send confirmation email after successful payment.
FR-08	Refund	ATTENDEE requests refunds; only ADMIN approves or rejects them.
FR-09	Dashboard	Provide organizer metrics for events, upcoming events, tickets sold and revenue.
FR-10	AI & Images	Provide EventMitra AI assistance and authorized event image upload.

Booking and payment workflow: a new booking is stored with PENDING status. Ticket availability is reduced after the requested quantity is validated. A Razorpay order is created for the booking amount. The booking becomes CONFIRMED only after the payment signature is successfully verified. The payment record is stored as SUCCESS, a receipt is generated and a confirmation email/audit record is sent.

2.2 NON-FUNCTIONAL REQUIREMENTS AND ENVIRONMENT

Non-Functional Requirements

Category	Requirement
Performance	The application should respond promptly for normal CRUD, browsing and booking operations.

Security	Passwords are stored using BCrypt; protected endpoints require authentication and role-based authorization.
Reliability	Transactional booking/payment logic should maintain consistent booking and payment state.
Usability	The React interface should provide clear navigation for event discovery, registration, login, booking and payment.
Maintainability	The backend is separated into controller, service, repository, entity, DTO, security and exception packages.
Scalability	The notification functionality is isolated as a separate .NET service, allowing independent evolution.
Data Integrity	MySQL foreign-key relationships connect bookings to attendees, events and tickets, and payments/receipts/refunds to their parent records.

Software Environment

- Java 21 and Spring Boot 3.3.5 for the main backend.
- Maven for Java dependency and build management.
- MySQL 8.x for relational persistence.
- React 19 with Vite for the frontend.
- .NET 8 ASP.NET Core for the notification microservice.
- Razorpay test/live-compatible integration through the Razorpay Java SDK.
- Spring Security 6 style configuration with stateless JWT authentication.
- Spring AI configured with an OpenAI-compatible Groq endpoint and Llama 3.3 70B.

Hardware / Development Requirements

The project can be developed and demonstrated on a modern Windows/Linux development machine with a multi-core processor, at least 8 GB RAM and sufficient storage for the Java, Node.js, .NET and MySQL development environments. A browser, Java JDK, Maven, Node.js/npm, MySQL Server, and .NET 8 SDK/runtime are required for local execution.

3. SYSTEM ARCHITECTURE

EventMitra follows a layered web architecture. React 19 and Vite form the presentation layer. Spring Boot 3.3.5 provides REST APIs and business services. Spring Data JPA/Hibernate persist data in MySQL. Spring Security provides JWT-based stateless authentication and role-based authorization. The backend also integrates Razorpay, the .NET 8 notification service, local event-image storage and a Spring AI chat controller.

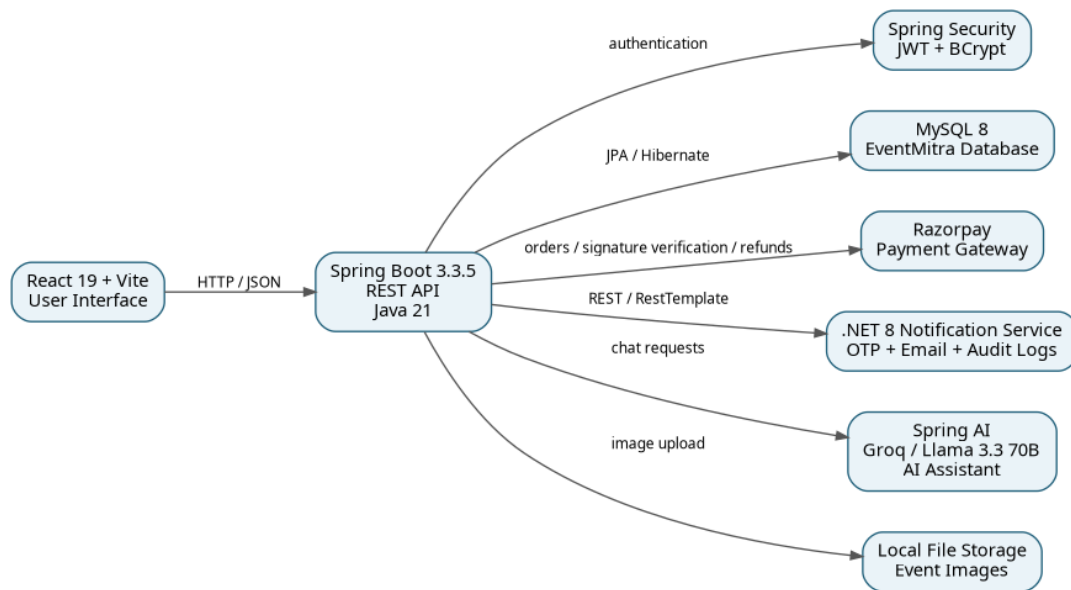


Fig. System Architecture of EventMitra

The separation of frontend, business logic, persistence and external integrations keeps the application modular and allows supporting services such as notification and AI assistance to evolve independently.

4. DIAGRAMS

4.1 ENTITY RELATIONSHIP DIAGRAM

The database design contains eight core tables. Users can act as organizers or attendees depending on role. An organizer creates events; events contain tickets; attendees create bookings against events and tickets. A booking can have a payment and a refund record, while successful payment generates a receipt. OTP verification records are associated with users.

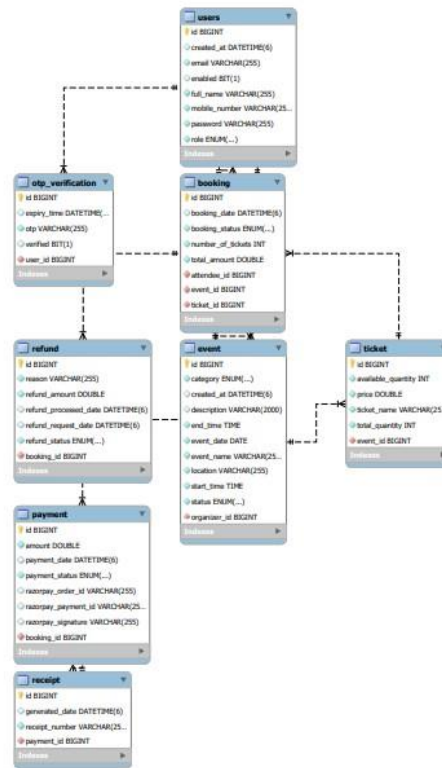


Fig. Entity Relationship Diagram for EventMitra (MySQL Workbench)

4.2 USE CASE DIAGRAM

The use case model identifies the three primary actors and their main interactions with EventMitra. Attendees focus on discovery, registration, booking, payment and refund requests. Organizers manage events and ticket inventory and monitor their dashboard. Administrators have the broadest management responsibilities, including refund approval and platform data management.

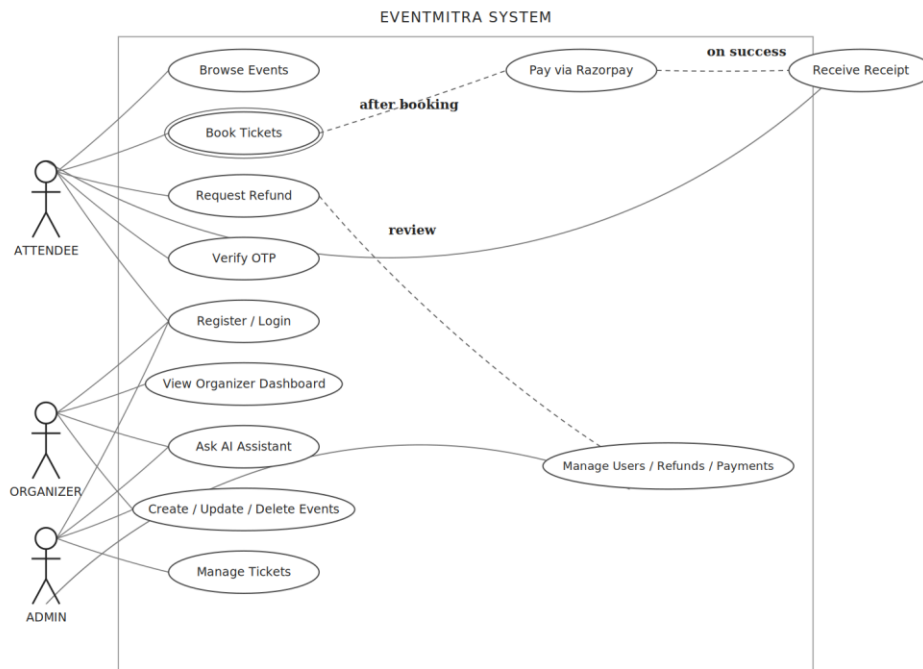


Fig. Use Case Diagram for EventMitra

4.3 DATA FLOW DIAGRAM

At a high level, the attendee, organizer and administrator interact with the EventMitra system. The system stores transactional information in MySQL and communicates with Razorpay for payments and the notification service for OTP, email and audit operations. AI chat requests are routed to the configured AI service.

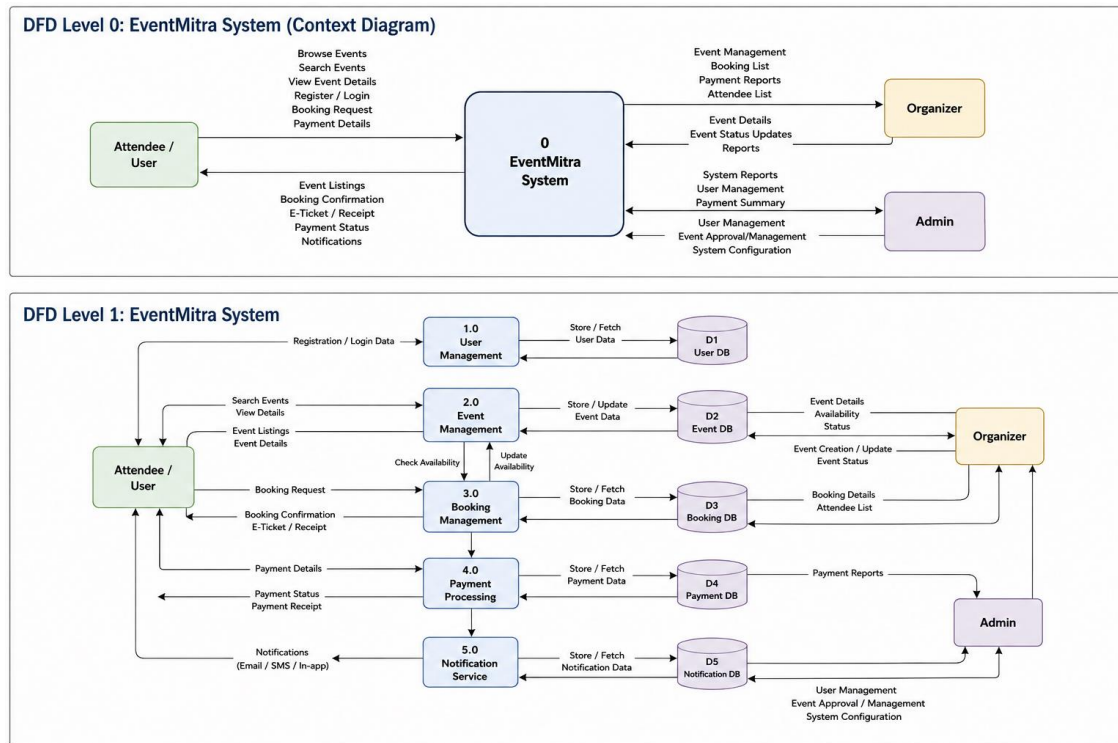


Fig. Data Flow Diagram

4.4 ACTIVITY DIAGRAM

The following activity flow describes the core attendee booking and payment process. It starts with authentication, continues through event and ticket selection, checks availability, creates a PENDING booking, creates a Razorpay order and completes payment verification. Only a valid payment signature leads to a CONFIRMED booking, receipt generation and notification.

1. Login Activity Diagram

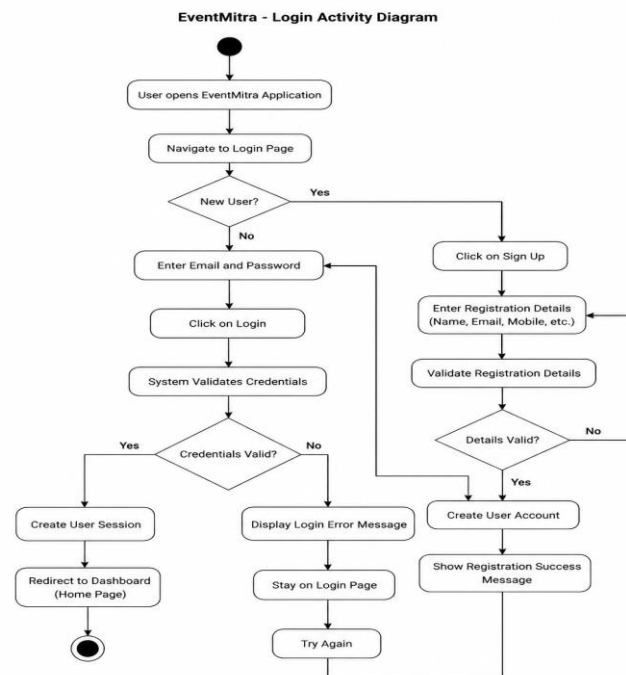
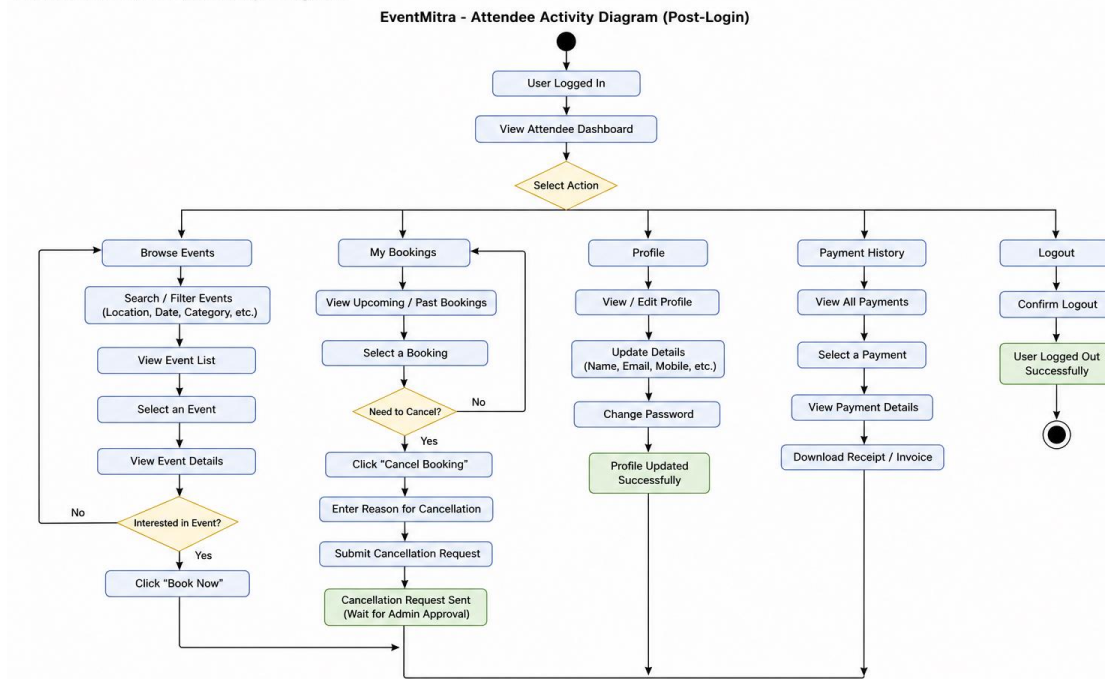
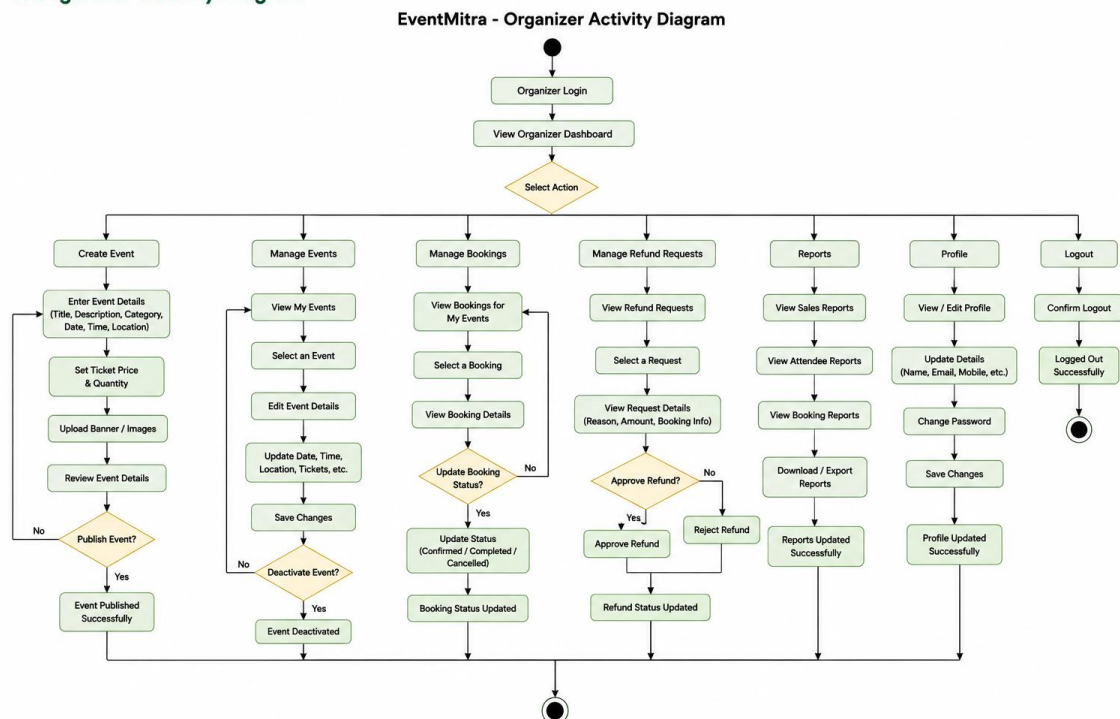


Fig. Activity Diagram for Login

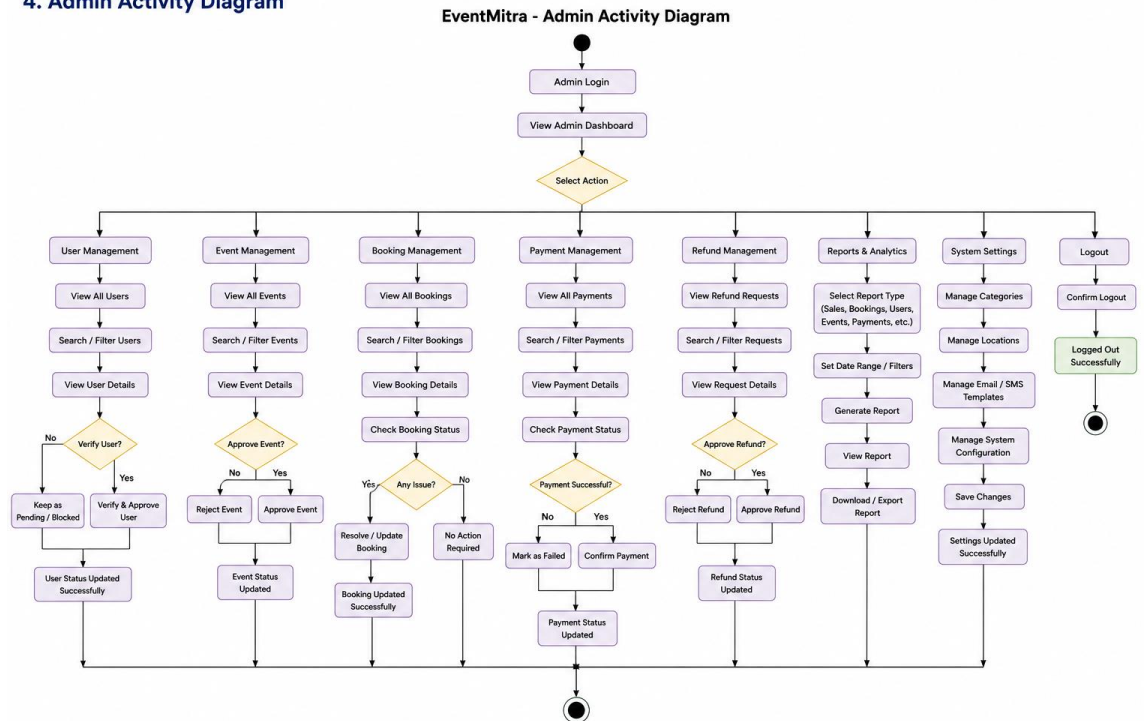
2. Attendee (User) Activity Diagram



3. Organizer Activity Diagram



4. Admin Activity Diagram



4.5 CLASS DIAGRAM

The class diagram reflects the main JPA entity model implemented in the backend. The model uses User, Event, Ticket, Booking, Payment, Receipt, Refund and OtpVerification classes. Relationships are mapped using JPA annotations such as @ManyToOne and @OneToOne.

EventMitra – Class Diagram

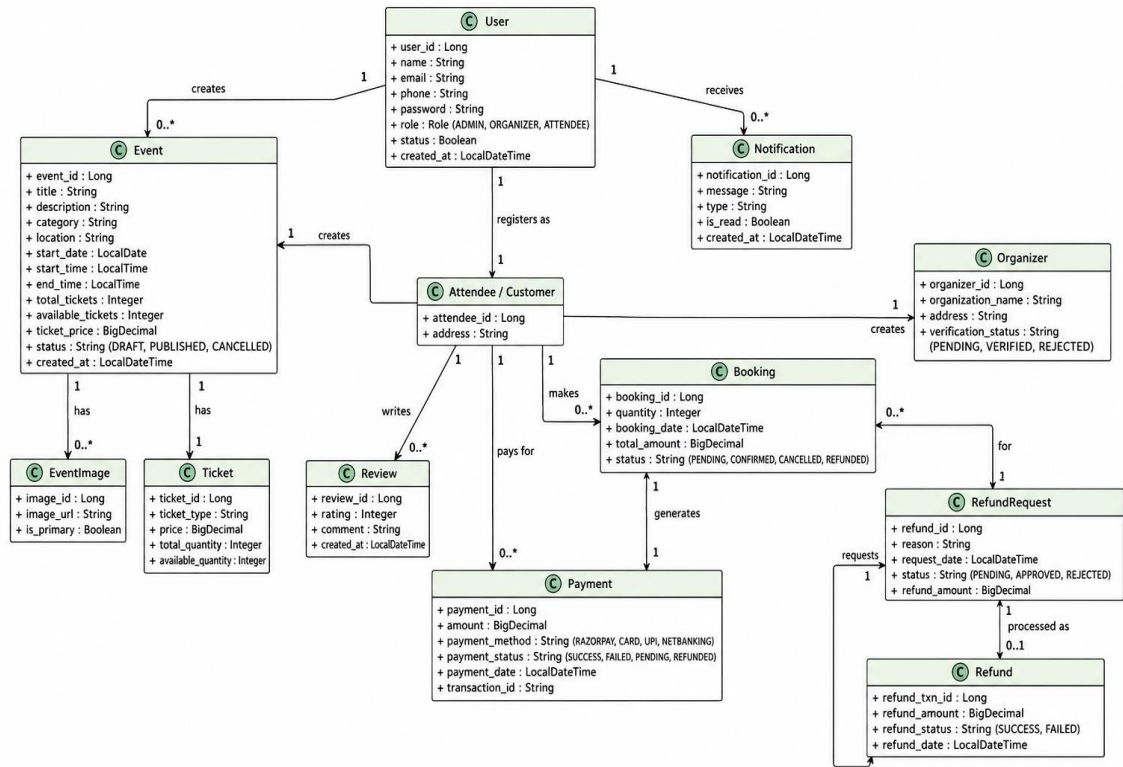


Fig. Class Diagram for EventMitra

4.6 SEQUENCE DIAGRAM

The sequence below focuses on a successful booking and payment. The React UI sends a booking request to Spring Boot. The service validates inventory and stores the booking. A Razorpay order is then created. After payment, the UI sends payment identifiers and signature to the verification endpoint. The backend validates the signature, persists the successful payment, confirms the booking, generates a receipt and triggers notification/audit operations.

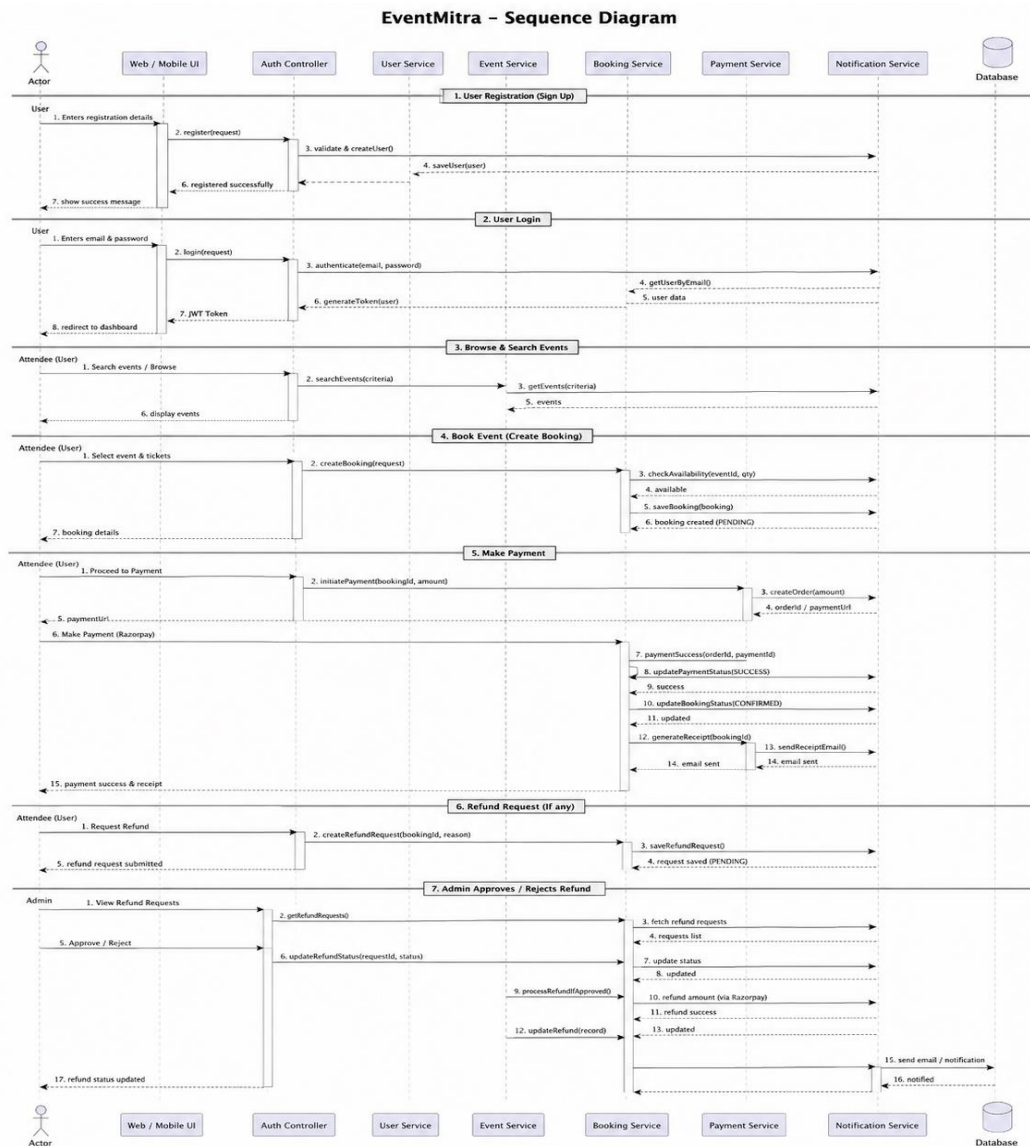


Fig. Sequence Diagram

5. DATABASE DESIGN

5.1 SCHEMA OVERVIEW

MySQL is the primary relational database for EventMitra. The supplied database design contains eight core application tables: users, event, ticket, booking, payment, receipt, refund and otp_verification. Primary keys use generated numeric identifiers, while foreign keys connect users, events, tickets and booking-related financial records.

Table	Purpose
users	Account, contact, password, role and activation state.
event	Event details and organizer ownership.
ticket	Ticket name, price and inventory.
booking	Attendee booking, quantity, amount and status.
payment	Payment status and Razorpay transaction identifiers.
receipt	Receipt number, date and payment link.
refund	Refund reason, amount and processing status.
otp_verification	OTP value, expiry, verification state and user link.

5.2 TABLE STRUCTURES – PART I

users

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Unique user identifier
full_name	VARCHAR(255)	NOT NULL	User full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
mobile_number	VARCHAR(25)	UNIQUE, NOT NULL	Registered mobile number
password	VARCHAR(255)	NOT NULL	BCrypt encoded password
role	ENUM	NOT NULL	ADMIN / ORGANIZER / ATTENDEE
enabled	BIT(1)	-	Account activation state
created_at	DATETIME(6)	-	Account creation time

event

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Event identifier
event_name	VARCHAR	NOT NULL	Event title
description	VARCHAR(2000)	-	Event description
event_date	DATE	NOT NULL	Event date
start_time	TIME	NOT NULL	Start time
end_time	TIME	NOT NULL	End time
location	VARCHAR(255)	NOT NULL	Event location
category	ENUM	NOT NULL	MUSIC / WORKSHOP / SPORTS / TECH / CONFERENCE / CULTURAL
status	ENUM	NOT NULL	UPCOMING / ONGOING / COMPLETED / CANCELLED
created_at	DATETIME(6)	NOT NULL	Creation timestamp
organizer_id	BIGINT	FK	User who created the event

ticket

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Ticket identifier
ticket_name	VARCHAR	NOT NULL	Ticket label
price	DOUBLE	NOT NULL	Ticket price

total_quantity	INT	NOT NULL	Total tickets
available_quantity	INT	NOT NULL	Tickets currently available
event_id	BIGINT	FK	Event to which ticket belongs

booking

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Booking identifier
booking_date	DATETIME(6)	-	Booking creation time
booking_status	ENUM	NOT NULL	PENDING / CONFIRMED / CANCELLED
number_of_tickets	INT	NOT NULL	Requested quantity
total_amount	DOUBLE	NOT NULL	Calculated booking amount
attendee_id	BIGINT	FK	Attendee user
event_id	BIGINT	FK	Booked event
ticket_id	BIGINT	FK	Selected ticket

5.3 DATABASE RELATIONSHIPS – PART II

payment

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Payment identifier
amount	DOUBLE	NOT NULL	Paid amount
payment_date	DATETIME(6)	-	Payment timestamp
payment_status	ENUM	NOT NULL	SUCCESS / FAILED / PENDING
razorpay_order_id	VARCHAR(255)	-	Razorpay order identifier
razorpay_payment_id	VARCHAR(255)	-	Razorpay payment identifier
razorpay_signature	VARCHAR(255)	-	Signature used for verification
booking_id	BIGINT	FK, One-to-One	Related booking

receipt

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Receipt identifier
receipt_number	VARCHAR(255)	UNIQUE	Unique receipt number
generated_date	DATETIME(6)	-	Receipt generation time
payment_id	BIGINT	FK, One-to-One	Related payment

refund

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	Refund identifier
reason	VARCHAR(255)	NOT NULL	Refund reason
refund_amount	DOUBLE	NOT NULL	Amount to refund
refund_request_date	DATETIME(6)	-	Request timestamp
refund_processed_date	DATETIME(6)	-	Approval/rejection time
refund_status	ENUM	NOT NULL	REQUESTED / APPROVED / REJECTED
booking_id	BIGINT	FK, One-to-One	Related booking

otp_verification

Column	Type	Key / Constraint	Description
id	BIGINT	PK, Auto Increment	OTP record identifier

otp	VARCHAR(255)	NOT NULL	OTP value
expiry_time	DATETIME(6)	-	OTP expiry time
verified	BIT(1)	-	Verification state
user_id	BIGINT	FK	User whose account is being verified

Relationship Summary

Relationship	Cardinality	Explanation
User → Event	1 : N	One organizer can create multiple events.
Event → Ticket	1 : N	An event can have multiple ticket records.
User → Booking	1 : N	An attendee can make multiple bookings.
Event → Booking	1 : N	An event can have multiple bookings.
Ticket → Booking	1 : N	A ticket record can be selected in multiple bookings.
Booking → Payment	1 : 1	A booking has one payment record in the implemented model.
Payment → Receipt	1 : 1	A successful payment generates one receipt.
Booking → Refund	1 : 1	A booking can have one refund request record.
User → OTP Verification	1 : N	OTP records are associated with a user.

6. API DESIGN AND SECURITY

Representative REST Endpoints

Module	Endpoint Examples	Access
Auth	POST /auth/register, /auth/login	Public
Events	GET /events; POST/PUT/DELETE /events	Read; write ADMIN/ORGANIZER
Tickets	GET /tickets; POST/PUT/DELETE /tickets	Read; write ADMIN/ORGANIZER
Bookings	POST /bookings; GET by id/attendee/event	Authenticated
Payments	POST create-order/verify; GET payments	Authenticated; list ADMIN
Refunds	POST request; PUT approve/reject	Request ADMIN/ATTENDEE; decision ADMIN
AI	POST /api/chat	Public

Security Implementation

- BCrypt protects stored passwords.
- JWT authentication is stateless and checked by JwtAuthenticationFilter.
- @PreAuthorize enforces role-based permissions on protected operations.
- Bean Validation protects request fields and unique user email/mobile constraints protect account data.
- Custom exceptions and GlobalExceptionHandler provide structured API errors.
- Razorpay payment signatures are verified before booking confirmation.

Payment Integrity

A booking starts as PENDING. A Razorpay order is created for its amount. A valid payment signature creates a SUCCESS payment, changes the booking to CONFIRMED and generates a receipt. Refund approval is restricted to ADMIN and, after a successful Razorpay refund, the booking is CANCELLED and ticket quantity is restored.

7. SNAPSHOTS

7.1 Database Tables

```
mysql> use eventmitra;
Database changed
mysql> show tables;
+-----+
| Tables_in_eventmitra |
+-----+
| __efmigrationshistory |
| auditlogs             |
| booking               |
| event                 |
| otp_verification      |
| otps                  |
| payment               |
| receipt               |
| refund                |
| ticket                |
| users                 |
+-----+
11 rows in set (0.02 sec)
```

```
mysql> table users;
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | created_at | email | role | enabled | full_name | mobile_number | password |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 2026-07-24 06:37:10.952823 | organizer@gmail.com | ORGANIZER | 0x01 | Rahul Sharma | 9999999992 | $2a$10$YgLEQrHoBkZsjNAh9VqyCeouAGYiLf62EbTqJd.PDjtuQxGVUTF6C |
| 2 | 2026-07-24 08:16:22.792478 | admin@gmail.com | ADMIN | 0x01 | Admin User | 9999999991 | $2a$10$2nILZf68tNm6mq0GUONt406HgHG8wEcTtpM/fa2yxEY0d6YZBHK60 |
| 3 | 2026-07-24 08:28:35.785329 | attendee@gmail.com | ATTENDEE | 0x01 | Rohit Kumar | 9999999993 | $2a$10$MCvXY3pYAswyyv9ayGJgQreeTtLtM29Rri6T02Bksw.8.97LMp4RE6 |
| 4 | 2026-07-26 09:32:09.787610 | yash123@gmail.com | ATTENDEE | 0x00 | Yash | 7655443245 | $2a$10$x4LczqeEPs8SuLne.70GZexlyFyGSI8ZNgLT0Ksabh5FvKY/5C. |
| 5 | 2026-07-26 10:54:18.831698 | yash456@gmail.com | ATTENDEE | 0x01 | Yash | 7655443267 | $2a$10$hNTazEy5RxxAnpwcFxJT.oJVPY1e8A0PhqQ0qTKl2ubGEBbiAKK |
| 6 | 2026-07-26 15:32:50.833739 | suryansh123@gmail.com | ATTENDEE | 0x01 | Suryansh | 7654367845 | $2a$10$x7F8IeuBgIFpApR004aWE0G0n07EXoFVx3d6LMTRH3CEXOKPu498m |
| 7 | 2026-07-27 11:36:22.588103 | manjiri123@gmail.com | ATTENDEE | 0x01 | Manjiri | 7388293831 | $2a$10$4Q/FXsXulIEG43IDEnPayOUZReIAeXndAZ0YhHqxU8Su0V7bR9VnK |
| 11 | 2026-08-05 15:05:21.212408 | gannaplus924@gmail.com | ADMIN | 0x00 | AdminTest | 7463847425 | $2a$10$HNP3F3Tk5yRF8cqTkwjszuXMjA8SH0pnh0FKJgc31Crh9fRnFJz6 |
| 12 | 2026-08-05 15:14:52.687880 | pacmantripathi@gmail.com | ADMIN | 0x00 | AdminTest | 7463847429 | $2a$10$Xwjpk7i2pY7QTckptia22uSzaJk9k1CymeFG9BewKXZFWqDZiSo.C |
| 13 | 2026-08-05 15:22:35.068294 | suryanshpratap Singh2022@gmail.com | ATTENDEE | 0x01 | Suryansh | 7463847426 | $2a$10$eUyRwdp00hP8DI/LmXL/QuY0ufvsYBF5gVYU1Hi6DY1TNeVIs0p7a |
| 14 | 2026-08-06 02:34:00.775146 | suryanshsingh6423@gmail.com | ATTENDEE | 0x01 | Suryansh(User) | 7583748283 | $2a$10$47AppCtAEo/vQGmAZaA0We0vXPCkkaWjHwPgZTBGZo32sc/89fZWi |
| 15 | 2026-08-06 03:51:49.872140 | gourimaslekar1521@gmail.com | ATTENDEE | 0x01 | Gouri Maslekar | 6537373985 | $2a$10$GK3ZUY13nCAnPNmm4mCmmOMQDJkZUF01hbJDKLKAPVoCQug3NX1v2
```

```
mysql> table ticket;
+-----+-----+-----+-----+-----+-----+
| id | available_quantity | price | ticket_name | total_quantity | event_id |
+-----+-----+-----+-----+-----+-----+
| 1 | 19 | 1400 | VIP | 100 | 1 |
| 2 | 899 | 150 | General | 1000 | 2 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.02 sec)
```

```
mysql> table booking;
```

id	booking_date	booking_status	number_of_tickets	total_amount	attendee_id	event_id	ticket_id
1	2026-07-26 11:03:10.338976	PENDING	1	1400	5	1	1
2	2026-07-26 11:11:22.050406	CONFIRMED	2	2800	5	1	1
3	2026-07-26 11:23:28.307749	CONFIRMED	2	2800	5	1	1
4	2026-07-26 11:40:37.523955	PENDING	1	1400	5	1	1
5	2026-07-26 12:49:13.120283	PENDING	1	1400	2	1	1
6	2026-07-26 15:33:49.055626	CONFIRMED	10	1500	6	2	2
7	2026-07-26 15:38:32.542458	PENDING	2	300	2	2	2
8	2026-07-27 11:34:25.339493	CONFIRMED	13	1950	5	2	2
9	2026-07-27 11:37:21.062920	CONFIRMED	3	4200	7	1	1
10	2026-07-30 07:27:40.336294	CONFIRMED	4	5600	5	1	1
11	2026-07-30 10:44:20.843782	CONFIRMED	9	1350	5	2	2
12	2026-07-31 15:34:50.120204	CONFIRMED	6	900	5	2	2
13	2026-08-06 02:03:09.345075	CONFIRMED	5	7000	13	1	1
14	2026-08-06 02:34:49.909602	CONFIRMED	8	1200	14	2	2
15	2026-08-06 02:40:21.099729	CONFIRMED	3	450	14	2	2
16	2026-08-06 03:52:41.586499	CONFIRMED	2	2800	15	1	1

16 rows in set (0.02 sec)

```
mysql> table event;
```

id	category	created_at	description	end_time	event_date	event_name
1	MUSIC	2026-07-24 06:48:31.552467	Live concert in Pune	22:00:00.000000	2026-08-20	Arijit Singh Live Concert Pune
2	TECH	2026-07-24 06:51:58.423469	National conference on AI and Machine Learning	17:30:00.000000	2026-09-15	AI Innovation Summit 2026 Delhi
3	SPORTS	2026-07-24 06:59:19.096242	HighSchool-college T20 Cricket Tournament	18:30:00.000000	2026-10-05	Pune Premier Music League Pune
5	CONFERENCE	2026-07-24 08:23:37.303711	Created by Admin	18:00:00.000000	2026-10-20	Admin Test Event Delhi
7	SPORTS	2026-07-27 11:39:44.190723	Cricket in Pune	19:11:00.000000	2026-08-05	Cricket Pune

5 rows in set (0.02 sec)

```
mysql> table auditlogs;
```

id	Action	Username	Details
1	USER_REGISTERED	suryanshsingh6423@gmail.com	User registered successfully.
2	BOOKING_CONFIRMED	suryanshpratap Singh2022@gmail.com	Booking ID 13 confirmed successfully for event Arijit Singh Live Concert
3	BOOKING_CONFIRMED	suryanshsingh6423@gmail.com	Booking ID 14 confirmed successfully for event AI Innovation Summit 2026
4	BOOKING_CONFIRMED	suryanshsingh6423@gmail.com	Booking ID 15 confirmed successfully for event AI Innovation Summit 2026
5	BOOKING_CONFIRMED	gourimaslekar1521@gmail.com	Booking ID 16 confirmed successfully for event Arijit Singh Live Concert

5 rows in set (0.02 sec)

```
mysql> table otp_verification;
```

id	expiry_time	otp	verified	user_id
1	2026-07-24 06:47:11.042713	863071	0x00	1
2	2026-07-24 08:26:22.880287	906319	0x00	2
3	2026-07-24 08:38:35.800974	893692	0x00	3
4	2026-07-26 09:42:09.835410	041596	0x00	4
5	2026-07-26 09:42:09.869072	589815	0x00	4
6	2026-07-26 11:04:18.901258	122739	0x00	5
7	2026-07-26 11:04:18.930582	471900	0x01	5
8	2026-07-26 15:42:59.981599	992440	0x00	6
9	2026-07-26 15:43:00.063093	154626	0x01	6
10	2026-07-27 11:46:22.604675	920976	0x00	7
11	2026-07-27 11:46:22.644413	358646	0x01	7

11 rows in set (0.01 sec)

```
mysql> table receipt;
```

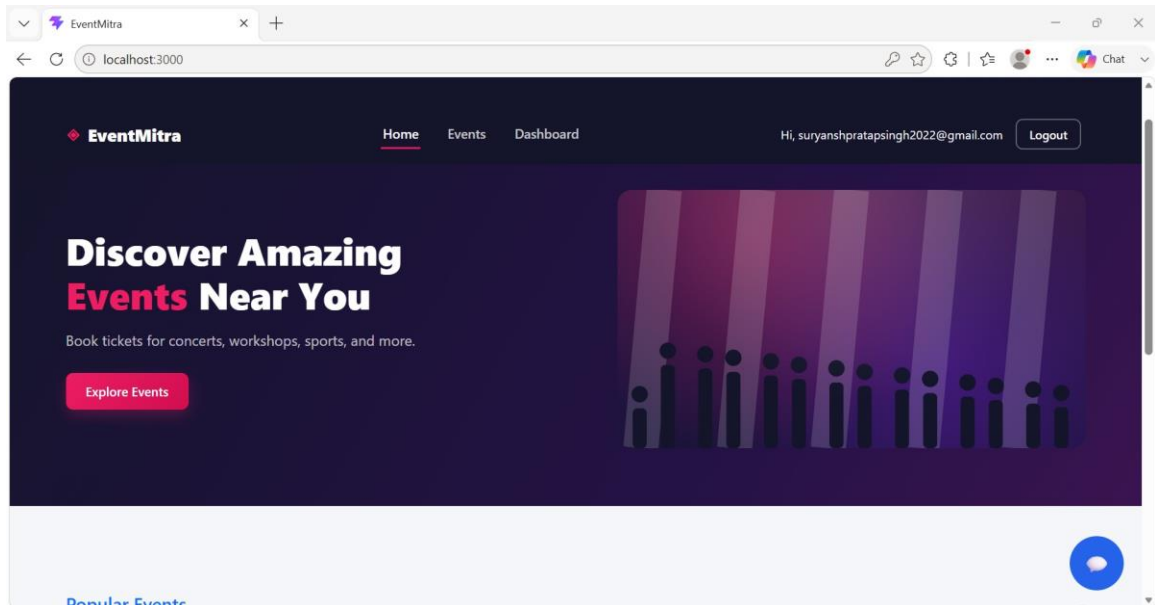
id	generated_date	receipt_number	payment_id
1	2026-07-26 11:18:07.258504	EM-10EF9EC3	1
2	2026-07-26 11:23:58.614357	EM-29ED7890	2
3	2026-07-26 15:34:22.646031	EM-9255D509	3
4	2026-07-27 11:35:07.933468	EM-A456327B	4
5	2026-07-27 11:37:46.125073	EM-CE291B9C	5
6	2026-07-30 07:28:22.838436	EM-015C4674	6
7	2026-07-30 10:44:51.005404	EM-6092CCDB	7
8	2026-07-31 15:35:26.461097	EM-20D09067	8
9	2026-08-06 02:11:39.834300	EM-C814917B	9
10	2026-08-06 02:35:21.106435	EM-3E48581E	10
11	2026-08-06 02:40:47.711183	EM-1FF402B5	11
12	2026-08-06 03:53:10.538704	EM-FBA8A19C	12

```
12 rows in set (0.02 sec)
```

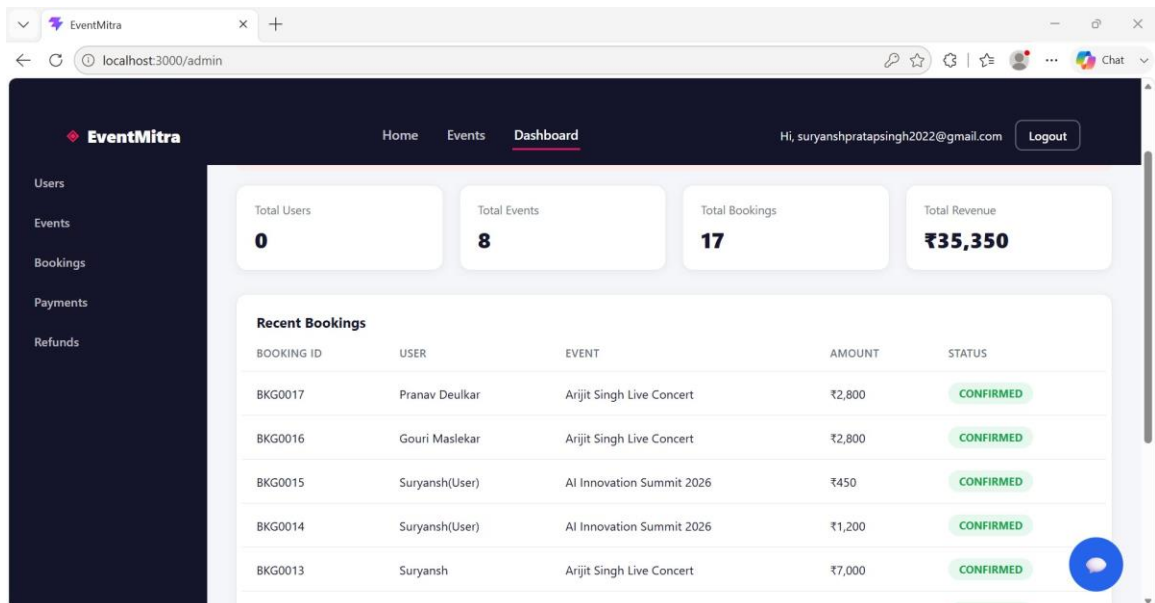
```
mysql> table payment;
```

id	amount	payment_date	payment_status	razorpay_order_id	razorpay_payment_id	razorpay_signature
1	2800	2026-07-26 11:18:07.242713	SUCCESS	order_TI6szGSMhEhKl5	pay_TI6zicljTAdSy	e95322b1024c8c4bd46071ddd2
2	2800	2026-07-26 11:23:58.609678	SUCCESS	order_TI75p4t8J8nvjK	pay_TI75whLBavIvus	f493317dd22921c3d7c13611fd
3	1500	2026-07-26 15:34:22.630104	SUCCESS	order_TIBMH65Z2WhfOD	pay_TIBMR8wZWBIGMO	3718fc928a9c0c6f00240dbc62
4	1950	2026-07-27 11:35:07.917695	SUCCESS	order_TIVoaGOGy3IBWw	pay_TIVolPubBQJXj	ffe393c5a3fe0919e4e510a502
5	4200	2026-07-27 11:37:46.125073	SUCCESS	order_TIVraQqVZX7AYV	pay_TIVrevMnEgneyA	7f6fab88d608b3c1a2b5936d23
6	5600	2026-07-30 07:28:22.806888	SUCCESS	order_TJdDFI1eH3hB99	pay_TJdDSsSteIfZaS	a37e54d8026b8eeeb43a622c72
7	1350	2026-07-30 10:44:50.990211	SUCCESS	order_TJgYwoUXKkfjx2	pay_TJgZ6W29y9KXWM	f2ee939a305f1835a12e98625f
8	900	2026-07-31 15:35:26.434258	SUCCESS	order_THA30CKZD3ZApL	pay_THA38tT3vU9e7d	74289bf19652c58ea3e660636e
9	7000	2026-08-06 02:11:39.728372	SUCCESS	order_TMJYhvkjK8agBc	pay_TMJYp5eZbbYLMg	d5d78c63cb9b49506c5a373894
10	1200	2026-08-06 02:35:21.097408	SUCCESS	order_TMJxh2bb4QCiaL	pay_TMJxocF5ie57a7	ea284b5b57b41ea32889f89020
11	450	2026-08-06 02:40:47.683313	SUCCESS	order_TMK3VvYRL6d0Ut	pay_TMK3c7V6Lfk99a	41df41618ceb67dcfbfd4f6937
12	2800	2026-08-06 03:53:10.515285	SUCCESS	order_TMLHwOGm4AI9Q6	pay_TMLI4MQt2k0qb1	a7928387ec31ce678ba72fbb6b

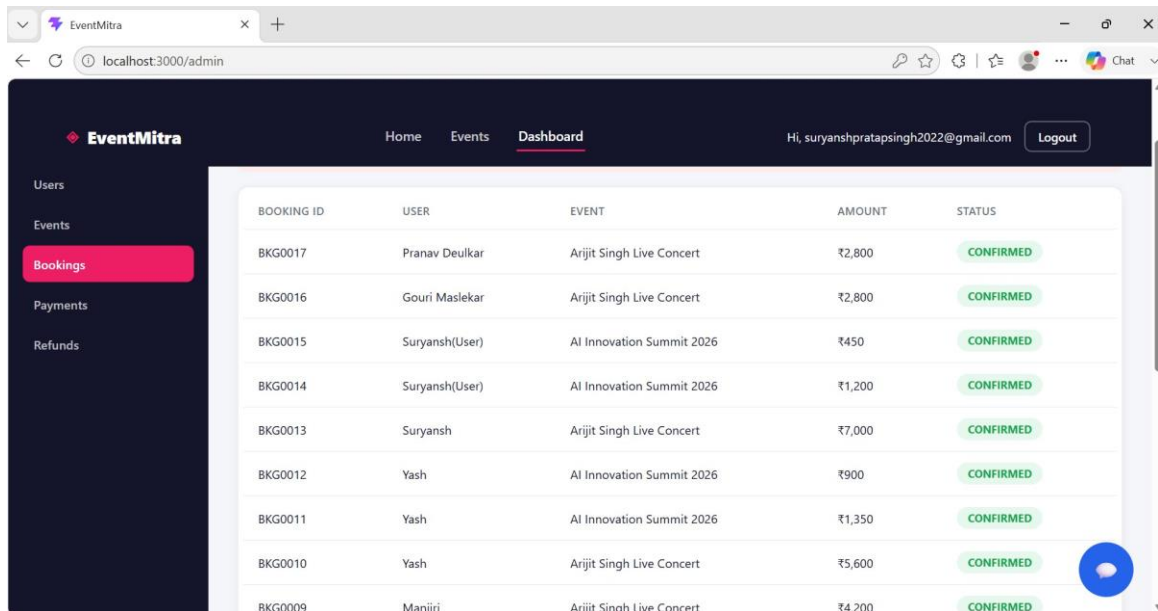
7.2 PROJECT SCREENSHOTS



Admin Login



Admin Dashboard



EventMitra

Home Events Dashboard

Hi, suryanshpratap Singh2022@gmail.com Logout

Users

Events

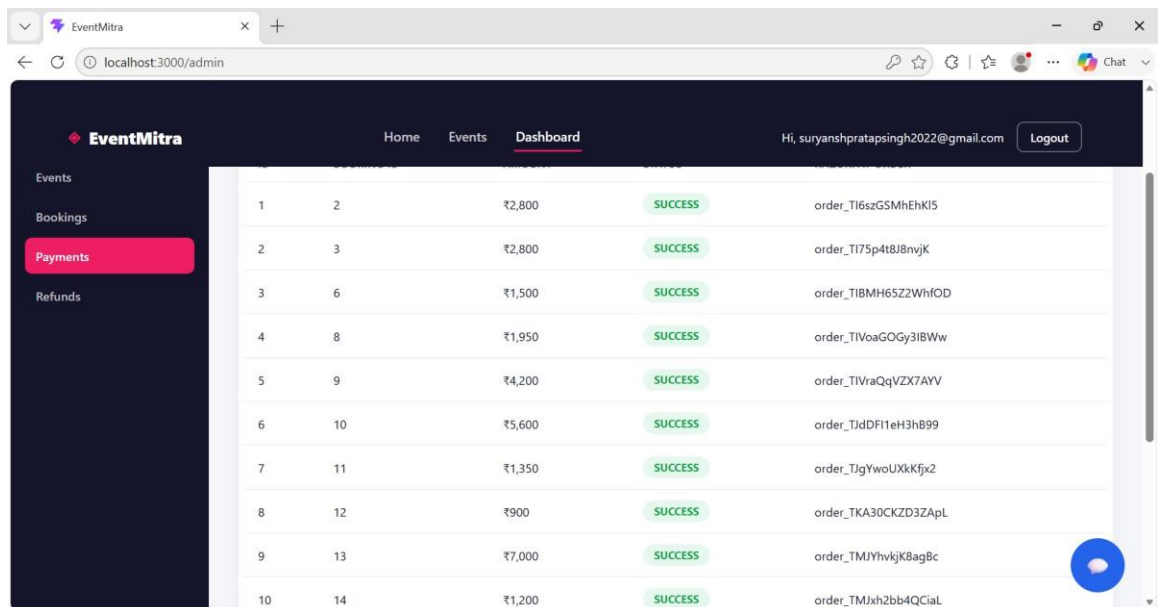
Bookings

Payments

Refunds

BOOKING ID	USER	EVENT	AMOUNT	STATUS
BKG0017	Pranav Deulkar	Arijit Singh Live Concert	₹2,800	CONFIRMED
BKG0016	Gouri Maslekar	Arijit Singh Live Concert	₹2,800	CONFIRMED
BKG0015	Suryansh(User)	AI Innovation Summit 2026	₹450	CONFIRMED
BKG0014	Suryansh(User)	AI Innovation Summit 2026	₹1,200	CONFIRMED
BKG0013	Suryansh	Arijit Singh Live Concert	₹7,000	CONFIRMED
BKG0012	Yash	AI Innovation Summit 2026	₹900	CONFIRMED
BKG0011	Yash	AI Innovation Summit 2026	₹1,350	CONFIRMED
BKG0010	Yash	Arijit Singh Live Concert	₹5,600	CONFIRMED
BKG0009	Manjiri	Arijit Singh Live Concert	₹4,200	CONFIRMED

Admin Dashboard



EventMitra

Home Events Dashboard

Hi, suryanshpratap Singh2022@gmail.com Logout

Events

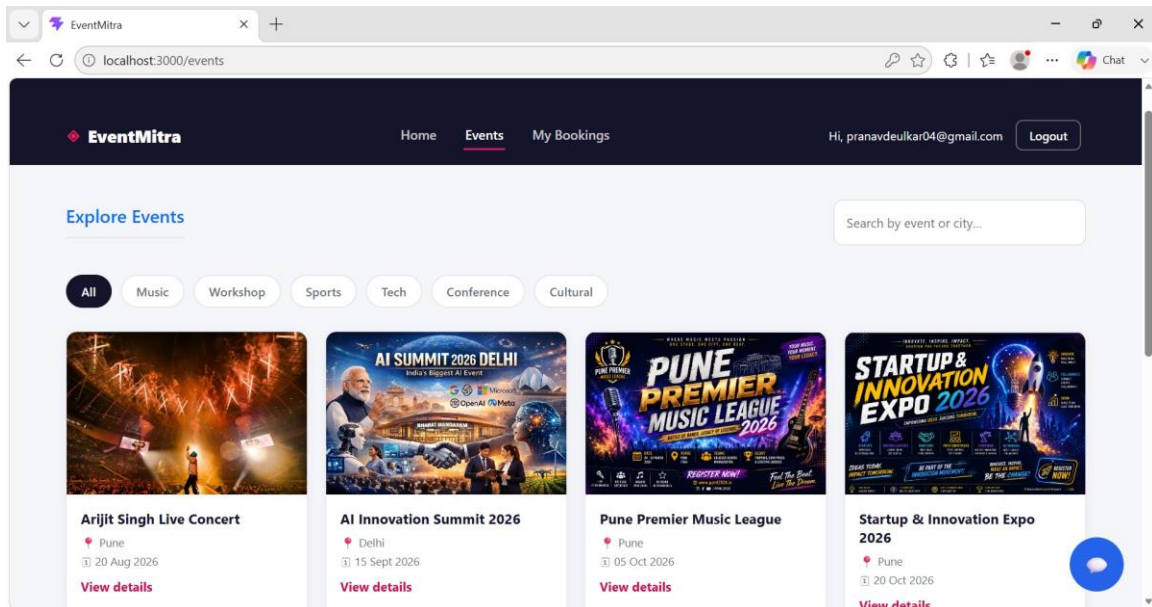
Bookings

Payments

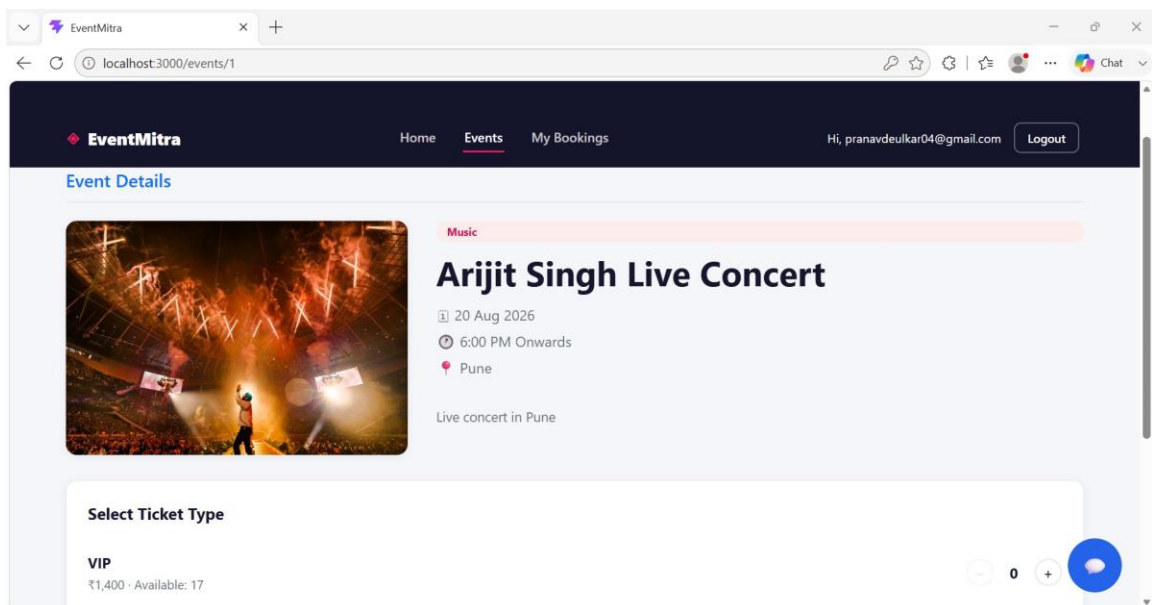
Refunds

1	2	₹2,800	SUCCESS	order_Ti6szGSMhEhKI5
2	3	₹2,800	SUCCESS	order_Ti7Sp4t8J8nvjK
3	6	₹1,500	SUCCESS	order_TiBMH65Z2WhfOD
4	8	₹1,950	SUCCESS	order_TiVoaGOGy3iBWw
5	9	₹4,200	SUCCESS	order_TiVraQqVZX7AVV
6	10	₹5,600	SUCCESS	order_TidDF1eH3hB99
7	11	₹1,350	SUCCESS	order_TigYwoUXkKfx2
8	12	₹900	SUCCESS	order_TKA30CKZD3ZApL
9	13	₹7,000	SUCCESS	order_TMjYhvkjK8agBc
10	14	₹1,200	SUCCESS	order_TMjXh2bb4QCiaL

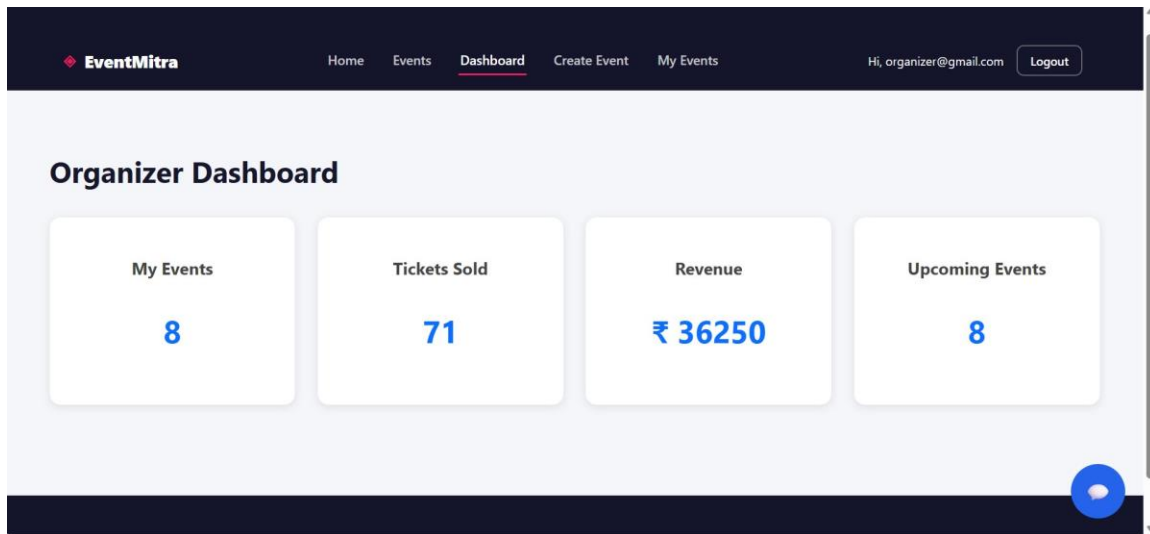
Admin Dashboard



User Login



User Event Booking



Organizer Dashboard

My Events

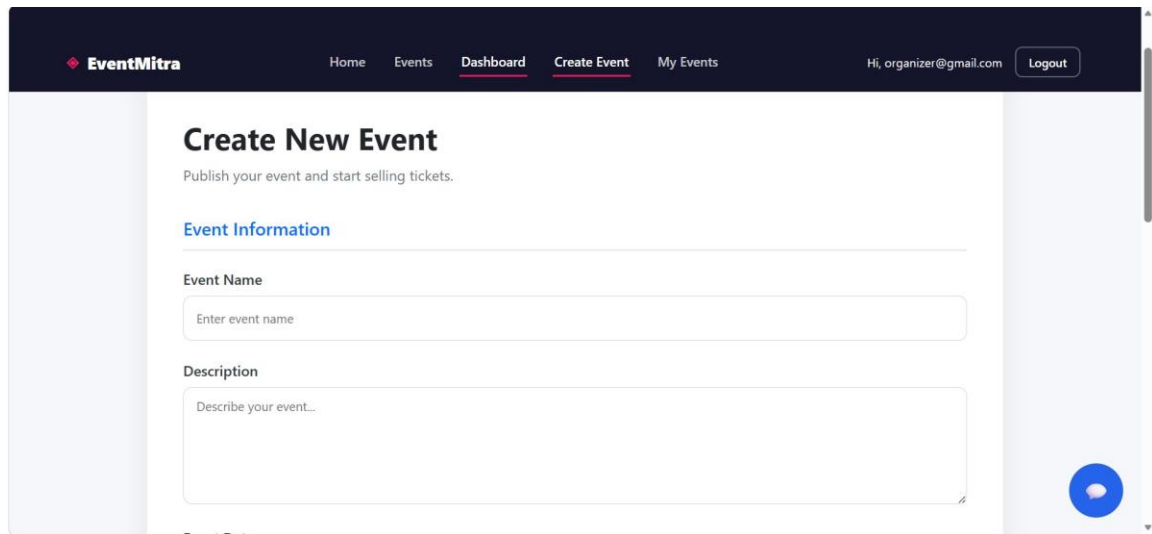
Manage all your events.

[+ Create Event](#)

Search event...

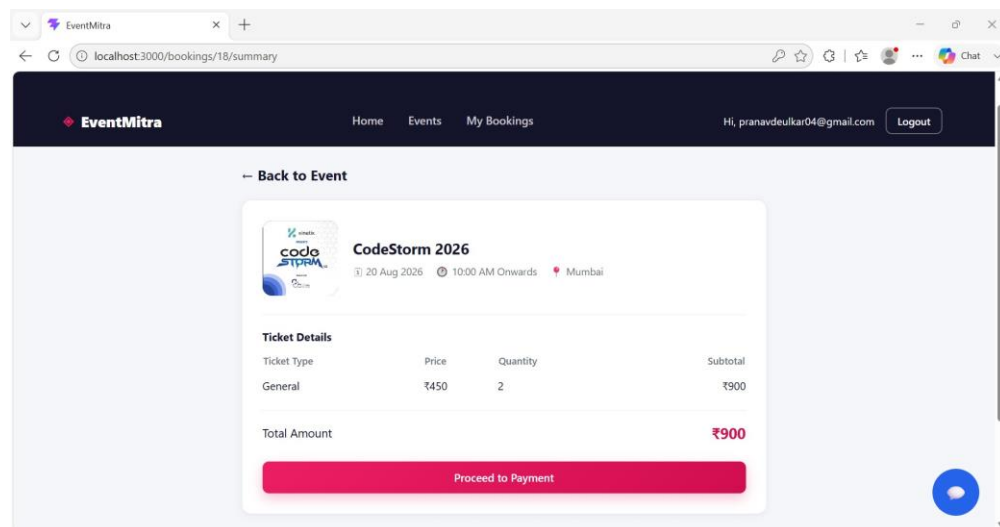
Event Name	Date	Location	Status	Actions
Arijit Singh Live Concert	2026-08-20	Pune	UPCOMING	Edit Tickets
AI Innovation Summit 2026	2026-09-15	Delhi	UPCOMING	Edit Tickets
Pune Premier Music League	2026-10-05	Pune	UPCOMING	Edit Tickets
Startup & Innovation Expo 2026	2026-10-20	Pune	UPCOMING	Edit Tickets
Cricket	2026-08-21	Pune	UPCOMING	Edit Tickets
CodeStorm 2026	2026-08-20	Mumbai	UPCOMING	Edit Tickets
Bollywood Beats Night	2026-08-30	Balewadi Stadium, Pune	UPCOMING	Edit Tickets
Inter-College Sports Fest 2026	2026-09-05	Shiv Chhatrapati Sports Complex, Pune	UPCOMING	Edit Tickets

Organizer MyEvents Page



Organizer Create New Event Page

7.3 BOOKING SCREENS



Ticket Type	Price	Quantity	Subtotal
General	₹450	2	₹900

Total Amount **₹900**

[Proceed to Payment](#)

Booking Screen

The screenshot shows the EventMitra payment gateway interface. At the top, there is a navigation bar with the EventMitra logo, links for Home, Events, and My Bookings, and a user profile section showing 'Hi, pranavdeulkar04@gmail.com' and a Logout button. The main content area is titled 'Payment' and contains a white card with the following details:

- Order Details**
 - Booking ID: BKG0018
 - Event: CodeStorm 2026
 - Amount: ₹900
- Select Payment Method**
 - A radio button is selected for 'Razorpay (Credit/Debit Card, UPI, Netbanking)'.
 - A green button labeled 'Pay ₹900 Now' is present.
 - Below the button, it says: 'You will be redirected to Razorpay secure gateway.' and '100% Secure Payments'.

A blue chat bubble icon is visible in the bottom right corner of the page.

Razorpay Payment Gateway

The screenshot shows the EventMitra payment integration interface in a web browser. The browser address bar shows 'localhost:3000/bookings/18/payment'. The interface is divided into two main sections:

- Price Summary (Left Panel):**
 - EventMitra logo.
 - Price Summary: ₹900.
 - User information: 'Using as +91 91345 67843' with a dropdown arrow.
 - A 'Secured by Razorpay' logo at the bottom.
- Payment Options (Right Panel):**
 - UPI:** Includes icons for Google Pay, PhonePe, and BHIM.
 - Cards:** Includes icons for Visa, Mastercard, and American Express.
 - Netbanking:** Includes icons for various Indian banks.
 - Wallet:** Includes icons for Paytm, Amazon Pay, and others.
 - Pay Later:** Includes icons for various credit and debit options.
 - All Wallet Options:** A dropdown menu showing 'Mobikwik' and 'Airtel Payments Bank' with right-pointing arrows.

A red banner in the top right corner indicates 'Test Mode'. A blue chat bubble icon is visible in the bottom right corner of the page.

Razorpay Payment Integration

EventMitra Home Events My Bookings Hi, pranavdeulkar04@gmail.com Logout

My Bookings

Payment successful — booking confirmed!

BOOKING ID	EVENT	DATE	TICKETS	AMOUNT	STATUS	ACTION
BKG0018	CodeStorm 2026	20 Aug 2026	2	₹900	Confirmed	<button>View</button> <button>Cancel</button>
BKG0017	Arijit Singh Live Concert	20 Aug 2026	2	₹2,800	Confirmed	<button>View</button> <button>Cancel</button>

User Bookings

EventMitra Home Events My Bookings Hi, pranavdeulkar04@gmail.com Logout

← Back to Event

CodeStorm 2026
20 Aug 2026 10:00 AM Onwards Mumbai

Ticket Details

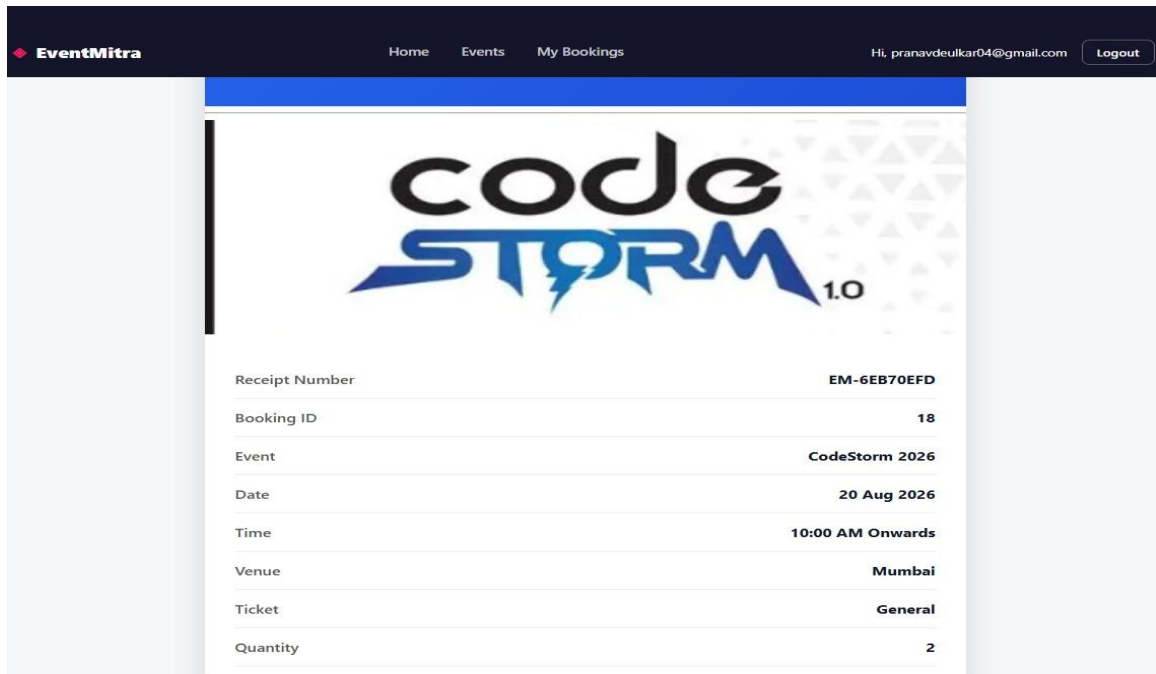
Ticket Type	Price	Quantity	Subtotal
General	₹450	2	₹900

Total Amount **₹900**

☒ This booking is already confirmed and paid.

View Receipt

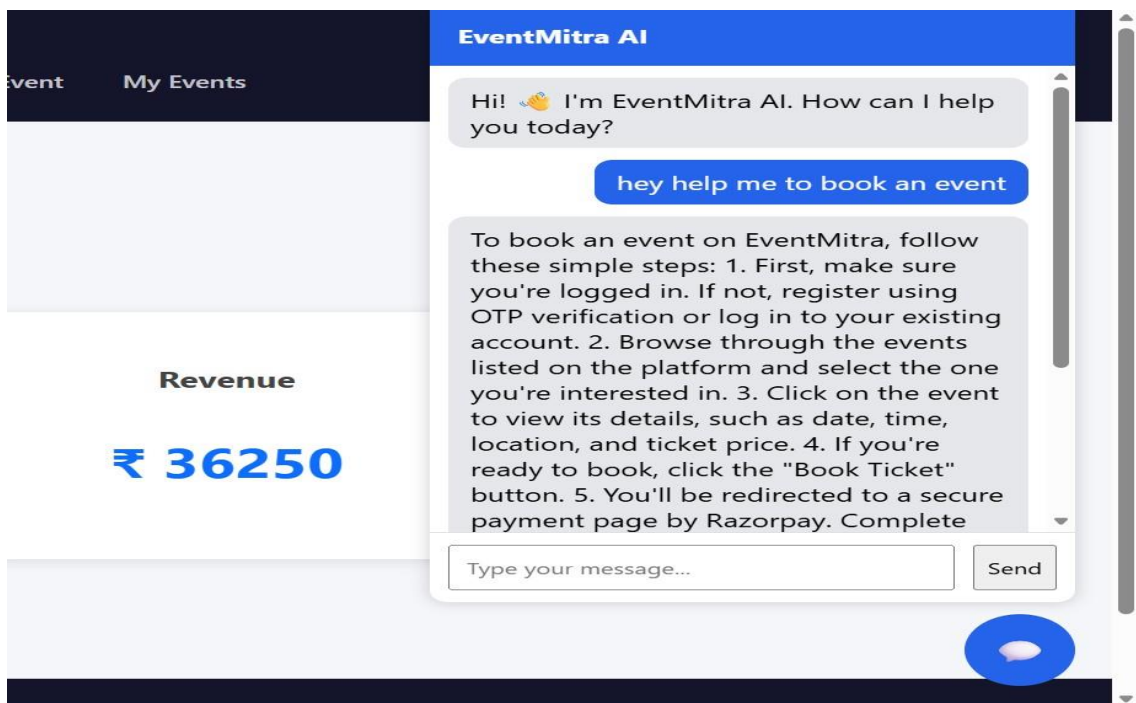
Printing Receipt



The image shows a receipt from EventMitra for the CodeStorm 1.0 event. The receipt is displayed on a website with a dark blue header containing the EventMitra logo and navigation links (Home, Events, My Bookings). The user is logged in as 'Hi, pranavdeulkar04@gmail.com' with a 'Logout' button. The receipt itself has a white background with a blue border and features the 'code STORM 1.0' logo at the top. Below the logo, the receipt details are listed in a table format.

Receipt Number	EM-6EB70EFD
Booking ID	18
Event	CodeStorm 2026
Date	20 Aug 2026
Time	10:00 AM Onwards
Venue	Mumbai
Ticket	General
Quantity	2

Receipt



The image shows the EventMitra AI chat interface. The chat window is titled 'EventMitra AI' and has a blue header. The chat history shows a greeting from the AI: 'Hi! 🤖 I'm EventMitra AI. How can I help you today?'. The user has responded with 'hey help me to book an event'. The AI has provided a detailed response with five steps to book an event: 1. Log in or register, 2. Browse events, 3. Select an event, 4. Click 'Book Ticket', and 5. Complete payment via Razorpay. The chat input field at the bottom contains the text 'Type your message...' and a 'Send' button. A blue circular button with a white speech bubble icon is located at the bottom right of the chat window.

Event

My Events

Revenue

₹ 36250

EventMitra AI

Hi! 🤖 I'm EventMitra AI. How can I help you today?

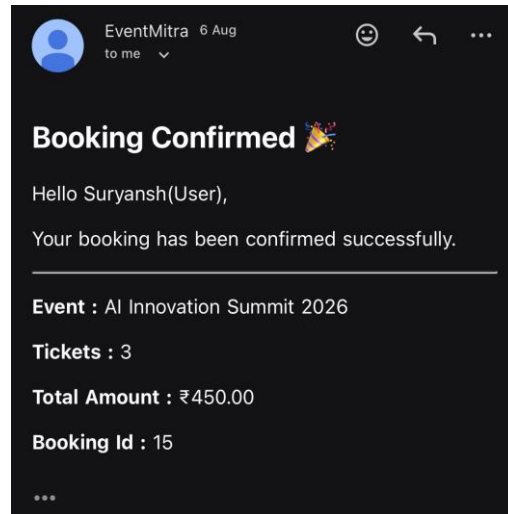
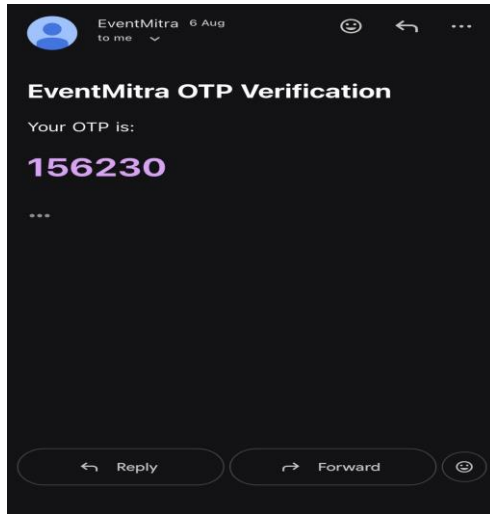
hey help me to book an event

To book an event on EventMitra, follow these simple steps: 1. First, make sure you're logged in. If not, register using OTP verification or log in to your existing account. 2. Browse through the events listed on the platform and select the one you're interested in. 3. Click on the event to view its details, such as date, time, location, and ticket price. 4. If you're ready to book, click the "Book Ticket" button. 5. You'll be redirected to a secure payment page by Razorpay. Complete

Type your message...

Send

EventMitra AI



Emails Received by EventMitra

8. TESTING AND VALIDATION

Testing was performed around the major workflows of EventMitra, with emphasis on authentication, role restrictions, event and ticket CRUD operations, ticket availability, payment verification and refund processing. The project also contains custom exceptions for missing resources, unavailable tickets, payment failures and invalid refund states.

Test ID	Scenario	Expected Result	Status
TC-01	Register with valid user details	User is created with disabled state and OTP request is initiated.	PASS
TC-02	Verify valid OTP	OTP is accepted and user is activated.	PASS
TC-03	Login with valid credentials	JWT token and user role are returned.	PASS
TC-04	Create event as organizer	Event is persisted successfully.	PASS
TC-05	Create event as unauthorized attendee	Request is rejected by authorization.	PASS
TC-06	Book more tickets than available	TicketUnavailableException is returned.	PASS
TC-07	Create Razorpay order	Order is created for booking amount.	PASS / integration dependent
TC-08	Verify valid payment signature	Payment saved as SUCCESS; booking becomes CONFIRMED; receipt generated.	PASS / gateway dependent
TC-09	Verify invalid payment signature	Payment is rejected and booking is not confirmed.	PASS
TC-10	Approve eligible refund as admin	Razorpay refund is initiated; booking cancelled; ticket quantity restored.	PASS / gateway dependent
TC-11	Approve refund as non-admin	Request is rejected by role authorization.	PASS
TC-12	Ask EventMitra AI assistant	Context-aware response is returned through chat endpoint.	PASS / AI service dependent

Exception and Validation Handling

`ResourceNotFoundException` is used when referenced users, events, tickets, bookings, payments or refunds do not exist. `DuplicateResourceException` protects unique user email/mobile constraints. `TicketUnavailableException` protects ticket inventory. `PaymentFailedException` handles payment order or signature failures, while `RefundNotAllowedException` protects invalid refund states. `GlobalExceptionHandler` converts these failures into API-friendly responses.

9. CONCLUSION

EventMitra demonstrates a complete event ticket booking and management platform combining React, Spring Boot, MySQL, Razorpay and a separate .NET notification service. The workflow connects registration, OTP verification, authentication, event discovery, ticket inventory, booking, payment, receipt generation and refund processing.

The project applies role-based access control, DTO validation, JPA relationships, custom exception handling and external-service integration. The database provides traceability between users, events, tickets, bookings, payments, receipts, refunds and OTP records.

Future enhancements can include cloud image storage, refresh-token management, advanced search, scheduled OTP cleanup, richer analytics and production deployment automation.

10. REFERENCES

1. Spring Boot documentation – <https://docs.spring.io/spring-boot/>
2. Spring Security reference – <https://docs.spring.io/spring-security/reference/>
3. Spring Data JPA documentation – <https://docs.spring.io/spring-data/jpa/>
4. Hibernate ORM documentation – <https://hibernate.org/orm/documentation/>
5. MySQL documentation – <https://dev.mysql.com/doc/>
6. Razorpay API documentation – <https://razorpay.com/docs/api/>
7. React documentation – <https://react.dev/>
8. Vite documentation – <https://vite.dev/>
9. ASP.NET Core documentation – <https://learn.microsoft.com/aspnet/core/>
10. Entity Framework Core documentation – <https://learn.microsoft.com/ef/core/>
11. Spring AI documentation – <https://docs.spring.io/spring-ai/reference/>
12. Project source code, database dump, ERD and UI screenshots supplied as part of the EventMitra project.